OPTIMISATION, CONTROL, AND DATA

M2 Mathématiques & Applications (Sorbonne Université)

From neural networks to neural ODEs

Machine learning · Feed-forward networks · Residual networks and control

Dario Prandi

CNRS, Université Paris-Saclay, CentraleSupélec

Slides for chapters 6–8 of the lecture notes · 2026

Outline

These slides cover chapters 6 to 8 of the lecture notes, with an epilogue on transformers. Numbered results follow the numbering of the notes, chapter by chapter.

Chapter 6 **Machine learning.**
Learning as nonlinear function approximation: data fitting, losses, generalization, a detour through SVMs.

Chapter 7 **Feed-forward neural networks.**
Architectures, learning by gradient descent and backpropagation, the universal approximation theorem.

Chapter 8 **Residual neural networks, neural ODEs and control.**
ResNets as Euler schemes, training as optimal control, expressivity as controllability (Tabuada and Gharesifard, without proof).

Epilogue **Large language models and transformers.**
Not in the notes. Attention, next-token prediction, transformers as interacting particle systems.

6

chapter

Machine learning

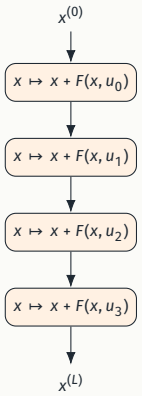
Why this part?

Machine learning is everywhere today: vision, language, science, engineering.

Its mathematical theory is still under construction:

- Why can a neural network approximate any function?
- Why does gradient descent work so well?
- What happens when the network becomes very deep?

Chapters 3 to 5 gave us the tools: numerical optimization, controllability, optimal control. This last part applies them to neural networks.



$L \to \infty$

$$\dot{x} = F(x, u(t))$$

$u(t)$: *control*
= weights of the network

Guiding thread: a deep network is the discretization of a control system.
Expressivity ↔ controllability, learning ↔ optimal control.

The fundamental problem: approximating a function

Let X, Y be two sets and $A(X, Y)$ a set of functions from X to Y. We are given a parametrized family

$$\mathcal{F} = \{\phi_\theta \in A(X, Y) \mid \theta \in \Theta\} \subset A(X, Y).$$

Problem 6.1 (machine learning problem)
Using the available information on $\psi \in A(X, Y)$, find $\hat{\theta} \in \Theta$ such that

$$\psi \approx \phi_{\hat{\theta}}.$$

- ψ is *unknown*: we only know some observations of it (e.g., pointwise values).
- Ingredients: the family $\mathcal{F}$ (the *model*), the information (the *data*), the criterion selecting $\hat{\theta}$ (the *loss*).

A classical problem: Fourier analysis, finite elements, linear regression...

Example 6.0.1: pointwise observations

Example 6.0.1 (linear approximation and pointwise observation)

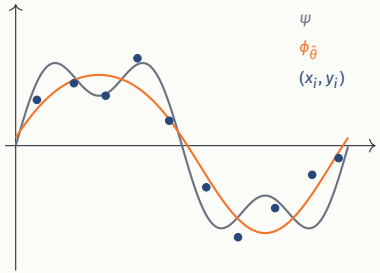
Let $\psi : [0, 2\pi] \to \mathbb{R}$ be continuous and $\{\phi_i\}_{i\geq 1}$ an orthonormal system of $L^2([0, 2\pi])$ (for instance $\cos(kx)$, $\sin(kx)$).

$$\mathcal{F} = \left\{\phi_\theta = \sum_{i=1}^{N} \theta_i \phi_i \;\middle|\; \theta \in \Theta = \mathbb{R}^N\right\}.$$

Information: M observations $y_i = \psi(x_i)$, $i \in [\![1, M]\!]$.

Criterion: least squares,

$$\hat{\theta} = \arg\min_{\theta\in\mathbb{R}^N} \sum_{i=1}^{M} \left|\phi_\theta(x_i) - y_i\right|^2.$$



A linear regression problem: $\theta \mapsto \phi_\theta$ is linear, so $\hat{\theta}$ solves a linear system.

Example 6.0.2: finite elements

Example 6.0.2 (finite elements)

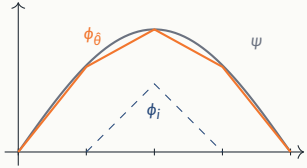
Let $\psi \in H_0^1(B_1(0))$, $B_1(0) \subset \mathbb{R}^d$, and $\{\phi_i\}_{i=1}^N \subset H_0^1$ be independent functions ("hat" functions on a mesh).

$$\mathcal{F} = \left\{ \phi_\theta = \sum_{i=1}^{N} \theta_i \phi_i \;\middle|\; \theta \in \mathbb{R}^N \right\}.$$

Information: the Laplacian $f = \Delta\psi$.

Criterion: minimize the energy $\left\| \phi_\theta - \psi \right\|_{H_0^1}$, that is,

$$\hat{\theta} = \arg\min_\theta \; \frac{1}{2} \int_{B_1(0)} \left\| \nabla \phi_\theta \right\|^2 \, \mathrm{d}x + \int_{B_1(0)} f \, \phi_\theta \, \mathrm{d}x.$$



- Same structure: parametrized family + information + criterion.
- Once again, $\theta \mapsto \phi_\theta$ is linear.

A common pattern, and what changes

In all these problems:

- an *unknown* $\psi : X \to Y$ to approximate.
- a *rich* family $\mathcal{F} = \{\phi_\theta\}_{\theta \in \Theta}$.
- an *optimal* $\hat{\theta}$, chosen from the available information.

“Modern” machine learning:

the map $\theta \mapsto \phi_\theta$ is nonlinear.

- we lose the normal equations, the Hilbert structure, uniqueness.
- we gain enormously in expressivity for a given number of parameters.

The data-fitting problem

Definition 6.0.1 (Data-fitting problem)

Given:

- a function $\psi : X \to Y$ of which we only know the sample $\mathfrak{X} = \{(x_i, \psi(x_i)) \mid i \in [\![1, M]\!]\}$.
- a family $\mathcal{F} = \{\phi_\theta : X \to Y \mid \theta \in \Theta\}$.
- a loss function $\mathcal{L} : Y \times Y \to \mathbb{R}$.

The problem (minimization of the empirical risk) is

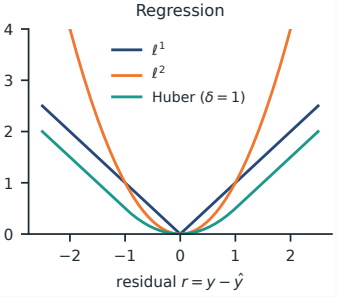
$$\text{find } \hat{\theta} \in \Theta \text{ such that} \quad \hat{\theta} = \arg\min_{\theta \in \Theta} \frac{1}{M} \sum_{i=1}^{M} \mathcal{L}\big(\phi_\theta(x_i), \psi(x_i)\big).$$

- The risk $\mathbb{E}_{x \sim \mu}\, \mathcal{L}(\phi_\theta(x), \psi(x))$, with μ the law of the inputs, is out of reach: we replace it by its empirical mean over the data.
- For neural networks, $\theta \mapsto \phi_\theta(x_i)$ is nonlinear: the problem is nonconvex in general.
- Three questions: **expressivity** (is $\mathcal{F}$ rich enough?), **optimization** (how to compute $\hat{\theta}$?), **generalization** (is $\phi_{\hat{\theta}} \approx \psi$ away from the data?).

Loss functions

Regression ($Y = \mathbb{R}^n$).
With the residual $r = y - \hat{y}$, take $\mathcal{L}(\hat{y}, y) = \ell(r)$:

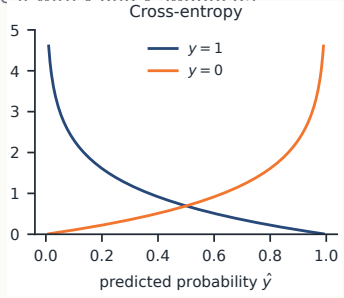
- ℓ^2: $\ell(r) = \|r\|_2^2$ (smooth).
- ℓ^1: $\ell(r) = \|r\|_1$ (robust, not smooth at 0).
- Huber: $\mathfrak{h}_\delta(r) = \frac{1}{2}\|r\|_2^2$ if $\|r\|_2 \le \delta$, else $\delta\|r\|_2 - \delta^2/2$.



Classification (y with values in $\{0, 1\}$: cat / dog).
Approximate by $\hat{y} \in (0, 1)$ (a probability) and use the cross-entropy

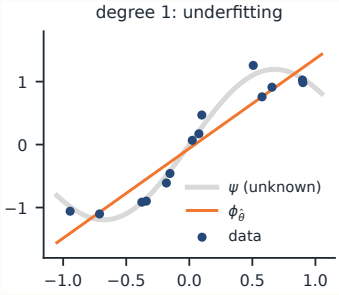
$$\mathcal{L}(\hat{y}, y) = -\big[y \log \hat{y} + (1 - y)\log(1 - \hat{y})\big],$$

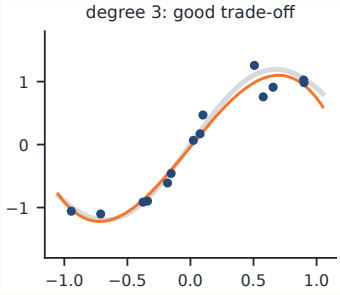
the "surprise" of observing y when predicting $\hat{y}$. (The notes write it with y and $\hat{y}$ swapped.)

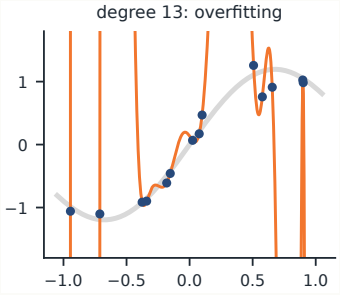


The real goal: generalization

We do not only want $\phi_{\hat{\theta}}(x_i) \approx \psi(x_i)$ on the data, but $\phi_{\hat{\theta}} \approx \psi$ everywhere.

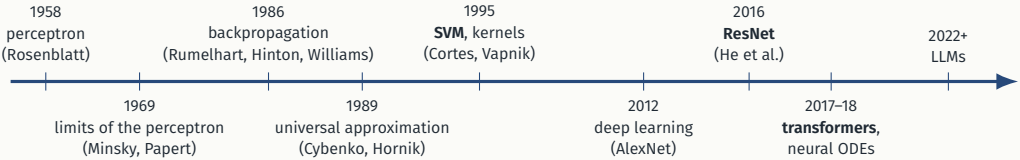






- A family too poor: *underfitting*. A family too rich: *overfitting*.
- Two distinct theoretical questions: expressivity (what can $\mathcal{F}$ represent?) and generalization (how good is $\phi_{\hat{\theta}}$ away from the data?). These slides focus on the first one.

A historical detour: support vector machines



- 1990s and 2000s: SVMs dominate. A clean theory (convex optimization, duality).
- Since 2012: deep networks dominate in practice, the theory runs behind.
- Why the detour? Understanding SVMs and their limits shows what neural networks bring.

This detour is not in the notes. Reference: [Ng, Ma, Chap. 6].

Linear classification and margin

Data $(x_i, y_i) \in \mathbb{R}^d \times \{-1, +1\}$, $i \in [\![1, M]\!]$.

Linear classifier: for $w \in \mathbb{R}^d$, $b \in \mathbb{R}$,

$$h_{w,b}(x) = \text{sign}(w^\top x + b).$$

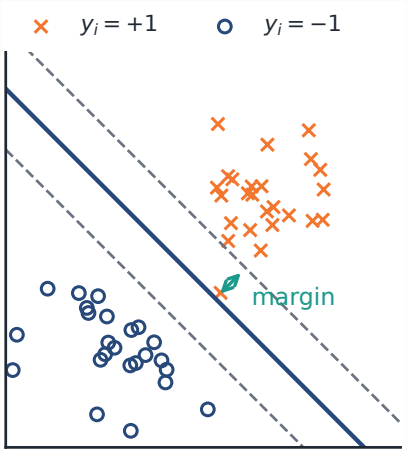
The data are separable if $y_i(w^\top x_i + b) > 0$ for every i.

Geometric margin:
the distance from x_i to the hyperplane $\{w^\top x + b = 0\}$,

$$\gamma_i = \frac{y_i\,(w^\top x_i + b)}{\|w\|_2} \quad \rightarrow \quad \text{margin} = \min_i \gamma_i.$$

A large margin = a *confident* prediction

(a small perturbation of the hyperplane does not change the label).



Idea [Vapnik]: among all separating hyperplanes, choose the one of maximal margin.

The maximal margin classifier

Up to rescaling (w, b), impose $\min_i y_i(w^\top x_i + b) = 1$. Then the margin is $1/\|w\|_2$, and

$$\min_{w,b} \frac{1}{2}\|w\|_2^2 \quad \text{subject to} \quad y_i(w^\top x_i + b) \geq 1, \quad i \in [\![1, M]\!].$$

→ A convex problem (quadratic with affine constraints): unique solution, efficiently computable.

Lagrange duality [Ng, Ma, §6.5; notes, Chap. 2]

The KKT conditions give $w = \sum_{i=1}^M \alpha_i y_i x_i$ with $\alpha_i \geq 0$, and the dual problem

$$\max_{\alpha \geq 0} \sum_{i=1}^M \alpha_i - \frac{1}{2}\sum_{i,j=1}^M \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle \quad \text{subject to} \quad \sum_i \alpha_i y_i = 0.$$

Only the $\alpha_i > 0$ matter: the support vectors (the points on the margin). $\implies h(x) = \operatorname{sign}\left(\sum_i \alpha_i y_i \langle x_i, x \rangle + b\right)$.

Everything depends on the data only through the inner products $\langle x_i, x_j \rangle$.

The kernel trick

Data that are not linearly separable? We embed them in a larger space,

$$\varphi : \mathbb{R}^d \to \mathcal{H} \quad \text{(the } \textit{feature map}\text{)},$$

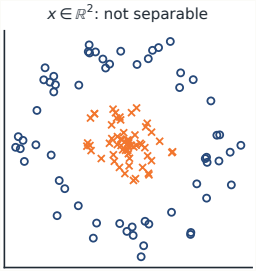
and run a linear SVM on the $\varphi(x_i)$.

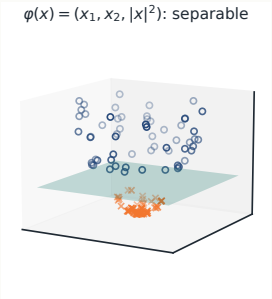
It is enough to know $\langle \varphi(x_i), \varphi(x_j) \rangle$, that is, the kernel

$$K(x, z) = \langle \varphi(x), \varphi(z) \rangle_{\mathcal{H}},$$

without ever computing φ!

Theorem [Mercer; Moore, Aronszajn]: every symmetric positive semi-definite kernel comes from some φ.





In $\mathbb{R}^2$: $\varphi(x) = (x_1, x_2, |x|^2)$ makes the rings separable by a plane.

Examples:

- polynomial: $K(x, z) = (1 + x^\top z)^p$.
- Gaussian: $K(x, z) = e^{-\|x-z\|^2/2s^2}$ ($\dim \mathcal{H} = \infty$).

Why is this limited?

What SVMs do well

- convex optimization: a global minimum is guaranteed.
- a solid statistical theory (VC dimension, margins).
- very effective for moderate M and good *features*.

Where it breaks down

- The kernel (hence φ) is chosen by hand: the representation is not learned.
- The family $\mathcal{F} = \{x \mapsto \sum_i \alpha_i K(x_i, x)\}$ is still linear in α: it is still linear approximation, in another space.
- Cost: the $M \times M$ Gram matrix. Out of reach for $M \sim 10^6$ to 10^9.
- Curse of dimensionality: for inputs such as images ($d \sim 10^6$), no generic kernel is adapted.

Neural networks learn the representation φ together with the classifier.
The price: a nonconvex optimization problem, and a much harder theory.

Chapter 6 in one slide

Take-away

- Machine learning = approximating an unknown ψ from a parametrized family $\mathcal{F} = \{\phi_\theta\}$, given information and a criterion.
- Classical examples (Fourier, finite elements, linear regression): $\theta \mapsto \phi_\theta$ is linear. Modern machine learning: nonlinear, more expressive, nonconvex.
- Data fitting (Definition 6.0.1): minimize the empirical risk $\frac{1}{M}\sum_i \mathcal{L}(\phi_\theta(x_i), \psi(x_i))$, with a loss adapted to regression or classification.
- Three questions: **expressivity** (what can $\mathcal{F}$ represent?), **optimization** (how to compute $\hat{\theta}$?), **generalization** (away from the data?).
- Support vector machines: maximal margin, convex duality, kernels. Clean theory, but φ is chosen by hand and the family stays linear in α.

Neural networks learn the representation. Next: expressivity (Chapter 7) and control (Chapter 8).

7

chapter

Feed-forward neural networks

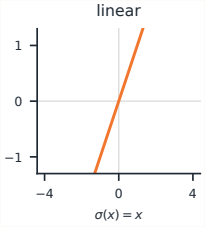
- Learning and backpropagation
- Universal approximation theorem

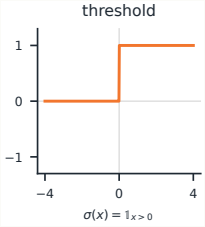
Nonlinearity is the key

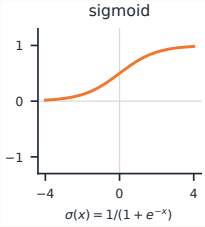
Building block: an affine map followed by a nonlinearity,

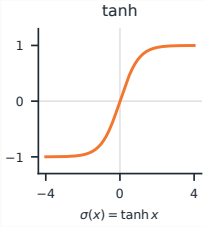
$$\phi_\theta(x) = \sigma(Wx + b), \qquad \theta = (W, b) \in \mathbb{R}^{n\times m} \times \mathbb{R}^n.$$

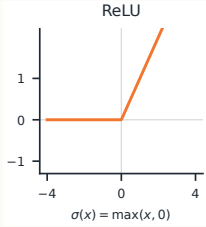
Here $\sigma : \mathbb{R} \to \mathbb{R}$ is the activation function, applied componentwise: $\sigma(z)_j := \sigma(z_j)$.











- Historically: the threshold (perceptron), then the sigmoid and tanh (smooth, bounded).
- Today: ReLU = $\max(x, 0)$ and its variants (GELU, softplus $\frac{1}{r}\log(1 + e^{rx})$).
- *NB.* Without σ, composing affine maps only gives an affine map.

One neuron, one hidden layer

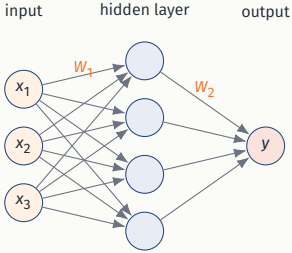
Definition 7.0.1 (One-hidden layer feed-forward ANN)
Let σ be an activation function and $d_1 \in \mathbb{N}$. A scalar network with one hidden layer $\phi_\theta : \mathbb{R}^d \to \mathbb{R}$ is given by

$$\begin{aligned}\phi_\theta(x) &= W_2\, \sigma(W_1 x + b_1) + b_2 \\ &= \sum_{j=1}^{d_1} (W_2)_j\, \sigma\Big(\sum_{i=1}^{d} (W_1)_{ji} x_i + (b_1)_j \Big) + b_2,\end{aligned}$$

with

$$\Theta = \big\{ (W_2, b_2, W_1, b_1) \in \mathbb{R}^{1\times d_1} \times \mathbb{R} \times \mathbb{R}^{d_1 \times d} \times \mathbb{R}^{d_1} \big\}.$$

- d_1 is the width of the hidden layer.
- W_ℓ: weight matrix, b_ℓ: bias of layer ℓ.
- number of parameters: $d_1(d+2)+1$.



Each hidden neuron computes $\sigma(w_j \cdot x + b_j)$: a *ridge function*, constant on the hyperplanes $w_j \cdot x = c$.

The XOR counterexample [Minsky, Papert 1969]

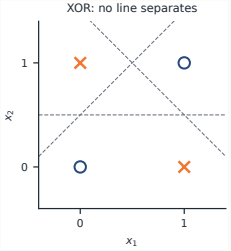
Four points, label $y = x_1 \oplus x_2$ (exclusive or):

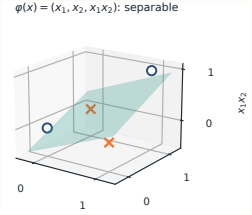
$$(0,0) \mapsto 0, \quad (1,1) \mapsto 0, \quad (0,1) \mapsto 1, \quad (1,0) \mapsto 1.$$

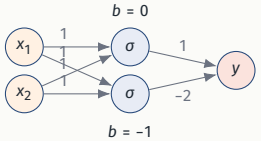
- No linear classifier works: if $w_1 x_1 + w_2 x_2 + b$ is < 0 at $(0,0)$ and $(1,1)$ and > 0 at $(0,1)$ and $(1,0)$, the sum of the first two inequalities contradicts the sum of the other two.
- This is the argument that ended the first wave of perceptrons.
- **With a kernel**: $\varphi(x) = (x_1, x_2, x_1 x_2)$ and the plane $x_1 + x_2 - 2x_1 x_2 = \frac{1}{2}$ separate. One had to *guess* φ.
- **With a network** with one hidden layer of two ReLU neurons:

  $\phi(x) = \sigma(x_1 + x_2) - 2\,\sigma(x_1 + x_2 - 1) = x_1 \oplus x_2$ on $\{0,1\}^2$.

  The nonlinearity builds the right feature.







Deep fully-connected networks

Definition 7.0.2 (Feed-forward fully-connected ANN with L hidden layers)

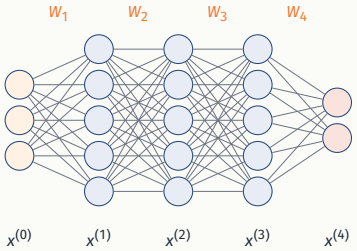
Let σ be an activation function and $d_1, \dots, d_L \in \mathbb{N}$. The network $\phi_\theta : \mathbb{R}^{d_0} \to \mathbb{R}^{d_{L+1}}$ is

$$\phi_\theta = \phi_{L+1} \circ \phi_L \circ \dots \circ \phi_1,$$

$$\phi_\ell(x^{(\ell-1)}) = \sigma(W_\ell x^{(\ell-1)} + b_\ell) =: x^{(\ell)}, \quad \ell \in [\![1, L]\!],$$

$$\phi_{L+1}(x^{(L)}) = W_{L+1} x^{(L)} + b_{L+1},$$

where $W_\ell \in \mathbb{R}^{d_\ell \times d_{\ell-1}}$, $b_\ell \in \mathbb{R}^{d_\ell}$, and the parameters are $\theta = (W_{L+1}, b_{L+1}, \dots, W_1, b_1)$.



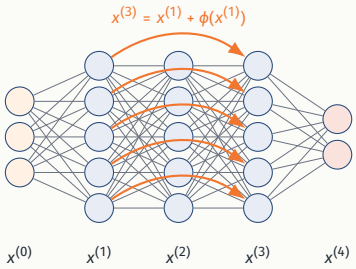
$L = 3$, $d_0 = 3$, $d_1 = d_2 = d_3 = 5$, $d_4 = 2$.

- L: depth, d_ℓ: width of layer ℓ, $x^{(\ell)}$: output of layer ℓ.
- $L = 1$: *shallow* network, $L > 1$: *deep* network.
- The last layer is affine (no activation on the output).

Variants of the architecture

- different activations in different layers, or even in different neurons.
- layer ℓ may depend on $(x^{(0)}, \dots, x^{(\ell-1)})$ and not only on $x^{(\ell-1)}$: skip connections, residual networks (ResNet). → Chapter 8.
- information may flow backwards: recurrent networks, with a time index.
- shared and structured weights: convolutional networks (images), attention (sequences, transformers). → Epilogue.

Basic mathematical questions:
(i) how to compute $\hat{\theta}$? **(ii)** what can $\mathcal{F} = \{\phi_\theta\}$ represent?



A skip connection: layer 3 sees $x^{(1)}$ directly.

7.1

Feed-forward neural networks

Learning and backpropagation

Learning = minimizing the empirical risk

We want to minimize

$$f(\theta) := \frac{1}{M}\sum_{i=1}^{M} \mathcal{L}\big(\phi_\theta(x_i), y_i\big), \qquad \theta \in \mathbb{R}^p.$$

Recall from chapter 3. $-\nabla f(\theta)$ is the direction of steepest descent.

Definitions 3.1.1 and 3.1.2 (recall)

Gradient flow:

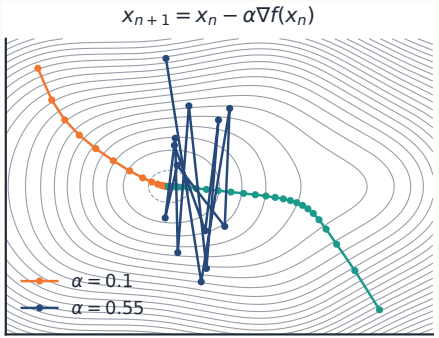
$$\begin{cases} \dot{\theta} = -\nabla f(\theta), \\ \theta(0) = \theta_0. \end{cases}$$

Gradient descent with step size (*learning rate*) $\alpha_n > 0$:

$$\theta_{n+1} = \theta_n - \alpha_n \nabla f(\theta_n).$$

In practice:

- p (number of parameters) and M (number of data) are very large.
- f is nonconvex.



The local minimum depends on θ_0. Too large a step: oscillations.

Why gradient descent works (recall from chapter 3)

Assume f is C^1 with L-Lipschitz gradient. Then

$$f(y) \le f(x) + \langle \nabla f(x), y - x \rangle + \frac{L}{2} \|y - x\|^2, \qquad \forall x, y.$$

With $y = \theta_{n+1}$, $x = \theta_n$ and $\alpha_n \le 1/L$:

$$f(\theta_{n+1}) \le f(\theta_n) - \alpha_n \Big(1 - \frac{\alpha_n L}{2}\Big) \|\nabla f(\theta_n)\|^2 \le f(\theta_n) - \frac{\alpha_n}{2} \|\nabla f(\theta_n)\|^2.$$

Guaranteed descent at every step, and $\sum_n \alpha_n \|\nabla f(\theta_n)\|^2 < \infty$ as soon as f is bounded from below.

Theorems 3.1.3 and 3.1.4 (recall)

If f is convex, coercive, with a unique minimizer $\theta^\star$, and $\sum \alpha_n = \infty$, $\alpha_n \le 1/L$, then $\theta_n \to \theta^\star$. Moreover, for $\alpha_n = \alpha$,

$$f(\theta_n) - f(\theta^\star) \le \|\theta_0 - \theta^\star\|^2 / 2\alpha n \qquad \text{(sublinear convergence)}.$$

Without convexity: the same proof gives $\nabla f(\theta_n) \to 0$, and every accumulation point is a critical point, not necessarily a minimum.

In practice one uses the stochastic version (SGD): the gradient is estimated on a subsample (*mini-batch*) [notes, §3.2].

The problem: computing the gradient

Gradient descent needs $\nabla_\theta \mathcal{L}(\phi_\theta(x), y)$, that is, the derivative of

$$\phi_\theta = \phi_{L+1} \circ \phi_L \circ \dots \circ \phi_1$$

with respect to each W_ℓ, b_ℓ.

- Chain rule $\rightsquigarrow$ a product of Jacobians of sizes $d_{k+1} \times d_k$:

$$\frac{\partial}{\partial W_\ell} \mathcal{L} = \frac{\partial \mathcal{L}}{\partial x^{(L+1)}} \frac{\partial x^{(L+1)}}{\partial x^{(L)}} \cdots \frac{\partial x^{(\ell+1)}}{\partial x^{(\ell)}} \frac{\partial x^{(\ell)}}{\partial W_\ell}.$$

- Naively, for each ℓ the whole product is recomputed: cost ~ $L\times$ (cost of one pass), and a huge memory if expanded (even for $\sigma = \mathrm{id}$, the sum has $d_1 \times \cdots \times d_L$ terms).
- With $p \sim 10^8$ to 10^{11} parameters, one needs an algorithm in $O(p)$ per data point.

Backpropagation [Rumelhart, Hinton, Williams 1986]: organize the chain rule *from the output to the input*, reusing the partial products. One forward pass and one backward pass suffice.

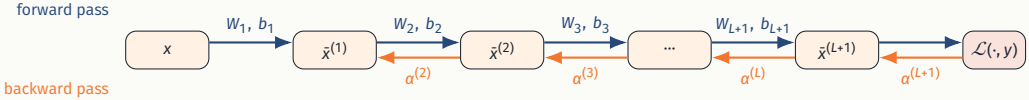
It is a special case of *reverse-mode automatic differentiation* and, as we shall see, of the *adjoint equation* of optimal control.

Backpropagation: forward and backward passes

Fix $x \in \mathbb{R}^{d_0}$, $y \in \mathbb{R}^{d_{L+1}}$. Define the pre-activations $\bar{x}^{(\ell)}$ and the adjoints $\alpha^{(\ell)} \in \mathbb{R}^{d_\ell}$ by

$$\begin{cases} \bar{x}^{(1)} = W_1 x + b_1, \\ \bar{x}^{(\ell)} = W_\ell \, \sigma(\bar{x}^{(\ell-1)}) + b_\ell, \quad \ell \in [\![2, L+1]\!], \end{cases} \qquad \begin{cases} \alpha^{(L+1)} = \dfrac{\partial}{\partial \bar{x}^{(L+1)}} \mathcal{L}(\bar{x}^{(L+1)}, y), \\ \alpha^{(\ell)} = \sigma'(\bar{x}^{(\ell)}) \odot (W_{\ell+1})^\top \alpha^{(\ell+1)}, \quad \ell = L, \dots, 1, \end{cases}$$

where $\odot$ is the componentwise product.



The forward pass computes and stores the $\bar{x}^{(\ell)}$. The backward pass computes the $\alpha^{(\ell)}$ from the output back to the input.

Backpropagation: the theorem

Theorem 7.1.1 (Backpropagation)

With the pre-activations $\bar{x}^{(\ell)}$ and the adjoints $\alpha^{(\ell)}$ of the previous slide,

$$\frac{\partial \mathcal{L}}{\partial b_\ell} = \alpha^{(\ell)}, \qquad \frac{\partial \mathcal{L}}{\partial W_1} = \alpha^{(1)} x^\top, \qquad \frac{\partial \mathcal{L}}{\partial W_\ell} = \alpha^{(\ell)}\, \sigma(\bar{x}^{(\ell-1)})^\top, \quad \ell \in [\![2, L+1]\!].$$

- The gradient with respect to the parameters of layer ℓ is an outer product: the adjoint $\alpha^{(\ell)}$ times the input of the layer.
- All the $\alpha^{(\ell)}$ come from a single backward recursion, which costs one matrix-vector product per layer.

Cost: one forward pass (compute and store the $\bar{x}^{(\ell)}$) + one backward pass (the $\alpha^{(\ell)}$) = about twice the cost of evaluating ϕ_θ, whatever the number p of parameters. Differentiating parameter by parameter would cost p evaluations.

Backpropagation: sketch of the proof Exercise

Lemma 7.1.2
$\alpha^{(\ell)} = \dfrac{\partial}{\partial \bar{x}^{(\ell)}} \mathcal{L}(\phi_\theta(x), y)$ for every $\ell \in [\![1, L+1]\!]$.

Proof. True for $\ell = L+1$ by definition. If it holds at rank $\ell+1$, the chain rule gives

$$\frac{\partial \mathcal{L}}{\partial \bar{x}^{(\ell)}} = \Big(\frac{\partial \bar{x}^{(\ell+1)}}{\partial \bar{x}^{(\ell)}}\Big)^\top \frac{\partial \mathcal{L}}{\partial \bar{x}^{(\ell+1)}} = \Big(\frac{\partial \bar{x}^{(\ell+1)}}{\partial \bar{x}^{(\ell)}}\Big)^\top \alpha^{(\ell+1)}, \qquad \Big(\frac{\partial \bar{x}^{(\ell+1)}}{\partial \bar{x}^{(\ell)}}\Big)_{ij} = (W_{\ell+1})_{ij}\, \sigma'(\bar{x}_j^{(\ell)}),$$

whence the recursion $\alpha^{(\ell)} = \sigma'(\bar{x}^{(\ell)}) \odot (W_{\ell+1})^\top \alpha^{(\ell+1)}$. ■

Proof of the theorem. $\bar{x}^{(k)}$ depends on (W_ℓ, b_ℓ) only for $k \geq \ell$, hence

$$\frac{\partial \mathcal{L}}{\partial W_\ell} = \frac{\partial \mathcal{L}}{\partial \bar{x}^{(\ell)}} \frac{\partial \bar{x}^{(\ell)}}{\partial W_\ell} = \alpha^{(\ell)} \frac{\partial \bar{x}^{(\ell)}}{\partial W_\ell}, \qquad \frac{\partial \bar{x}_k^{(\ell)}}{\partial (W_\ell)_{ij}} = \delta_{ki}\, \sigma(\bar{x}_j^{(\ell-1)}) \implies \frac{\partial \mathcal{L}}{\partial W_\ell} = \alpha^{(\ell)} \sigma(\bar{x}^{(\ell-1)})^\top.$$ ■

For $\ell = 1$ replace $\sigma(\bar{x}^{(0)})$ by x, and $\partial \bar{x}_k^{(\ell)} / \partial (b_\ell)_i = \delta_{ki}$ gives the formula for b_ℓ.

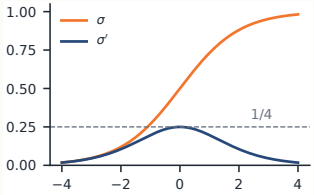
Exercises. Fill in the details of both proofs. Count the operations and the memory of one forward and one backward pass.

Two difficulties of deep learning [notes, §7.1.1]

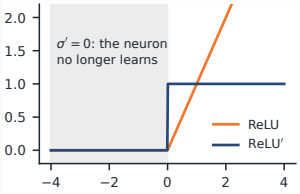
Vanishing gradient. If $|\sigma'| \le c < 1$ (sigmoid: $c = 1/4$), the recursion gives

$$\left\|\alpha^{(\ell)}\right\| \le c^{\,L+1-\ell}\ \left\|W_{\ell+1}\right\| \cdots \left\|W_{L+1}\right\|\ \left\|\alpha^{(L+1)}\right\|.$$

For large L, the gradients of the first layers are exponentially small: they stop learning.



Dead neurons. If $\sigma'(x) = 0$ for $x < a$ (ReLU: $a = 0$), a neuron whose pre-activation stays $< a$ has a zero gradient: its weights never move again. The neuron is dead.



Both problems come from the multiplicative structure of the chain rule.

Remedy: change the architecture (residual networks, chapter 8).

7.2

Feed-forward neural networks

Universal approximation theorem

Universal approximation: the definition

We want to make precise the idea that one can learn any function.

Definition 7.2.1 (Universal approximator)

A set $\mathcal{H}$ of functions $\mathbb{R}^d \to \mathbb{R}$ is a universal approximator of $C^0(\mathbb{R}^d)$ if for every $\varepsilon > 0$, every compact $K \subset \mathbb{R}^d$ and every $f \in C^0(\mathbb{R}^d)$ there exists $g \in \mathcal{H}$ such that

$$\|f - g\|_K := \sup_{x \in K} |f(x) - g(x)| \leq \varepsilon.$$

In other words, $C^0(\mathbb{R}^d) \subset \overline{\mathcal{H}}$ for uniform convergence on compact sets.

A minimal theoretical guarantee:

- It is the analogue of the Weierstrass theorem for polynomials: the family $\mathcal{F}$ is not a limiting factor.
- It says nothing about the size of the network needed, nor about the possibility of finding θ by optimization, nor about generalization.
- But without it, there is no hope of approximating ψ: it is a common-sense *necessary* condition.

The theorem for shallow networks

Definition 7.2.2 (Neural network spaces)

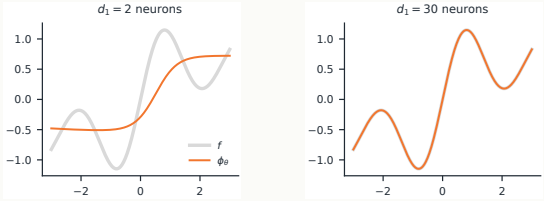
$\mathcal{N}_d^m(\sigma; L, n)$: the functions $\mathbb{R}^d \to \mathbb{R}^m$ realized by a network of depth exactly L, width at most n, activation σ. And $\mathcal{N}_d^m(\sigma; L) = \bigcup_{n \in \mathbb{N}} \mathcal{N}_d^m(\sigma; L, n)$.

Theorem 7.2.1 (Universal approximation for shallow neural networks) [Leshno et al. 1993]

Let σ be bounded and piecewise continuous (locally finitely many discontinuity points). Then

$$\mathcal{N}_d^1(\sigma; 1) \text{ is a universal approximator of } C^0(\mathbb{R}^d) \iff \sigma \text{ is not a polynomial.}$$

- If σ is a polynomial of degree p, every ϕ_θ is a polynomial of degree $\leq p$
  $\rightsquigarrow$ no universal approximation. (*Exercise.*)
- The nonlinearity is therefore essential.
- The theorem extends to other spaces (L^p) and other norms [Petersen, Zech 2025].



Least squares on d_1 tanh neurons with random weights.

Strategy of the proof

The hard part is $\Leftarrow$. It rests on three facts:

1. If $\mathcal{N}_1^1(\sigma; 1)$ is a universal approximator of $C^0(\mathbb{R})$, then $\mathcal{N}_d^1(\sigma; 1)$ is one of $C^0(\mathbb{R}^d)$.

   (reduction to dimension one)

2. If $\sigma \in C^\infty(\mathbb{R})$ is not a polynomial, then $\mathcal{N}_1^1(\sigma; 1)$ is a universal approximator of $C^0(\mathbb{R})$.

   (the smooth case)

3. If σ is bounded and piecewise continuous, $\mathcal{N}_1^1(\sigma; 1)$ approximates a smooth nonpolynomial activation $\tilde{\sigma}$.

   (regularization)

The essential tool:

Theorem 7.2.2 (Stone–Weierstrass)
Let $K \subset \mathbb{R}^d$ be compact and $\mathcal{H} \subset C^0(K, \mathbb{R})$ be such that

- for every $x \in K$ there is $f \in \mathcal{H}$ with $f(x) \neq 0$.
- for all $x \neq y$ in K there is $f \in \mathcal{H}$ with $f(x) \neq f(y)$.
- $\mathcal{H}$ is an algebra (stable under sums, products, multiplication by scalars).

Then $\mathcal{H}$ is dense in $C^0(K, \mathbb{R})$. In particular the polynomials are dense in $C^0(K)$.

Step 1: reduction to dimension one

Lemma 7.2.3
Let $\mathcal{H}$ be a universal approximator of $C^0(\mathbb{R})$ and $\mathcal{X} = \operatorname{span}\{x \mapsto g(w \cdot x) \mid w \in \mathbb{R}^d,\ g \in \mathcal{H}\}$. Then $\mathcal{X}$ is a universal approximator of $C^0(\mathbb{R}^d)$.

Proof. By Stone–Weierstrass it suffices to approximate the homogeneous polynomials of each degree k, $\mathcal{P}_k = \operatorname{span}\{x^\alpha \mid |\alpha| = k\}$ (written $\mathcal{H}_k$ in the notes, not to be confused with $\mathcal{H}$).

- By contradiction: if $\overline{\mathcal{X}} \cap \mathcal{P}_k \subsetneq \mathcal{P}_k$, there is a linear form $\mathfrak{q} \neq 0$ on $\mathcal{P}_k$ vanishing on $\overline{\mathcal{X}} \cap \mathcal{P}_k$.
  (Hahn–Banach)
- The D^α, $|\alpha| = k$, form a basis of the dual $\mathcal{P}_k'$ (since $D^\alpha x^\beta = \alpha!\,\delta_{\alpha\beta}$). Hence $\mathfrak{q} = \sum_{|\alpha|=k} q_\alpha D^\alpha$.
  (here $D^\alpha p(x) := \partial_{x_1}^{\alpha_1} \cdots \partial_{x_d}^{\alpha_d} p(x)$)
- The polynomial $p_w(x) = (w \cdot x)^k$ is in $\overline{\mathcal{X}}$ (universality of $\mathcal{H}$ for $d = 1$, composed with $w \cdot x$). But

$$p_w(x) = \sum_{|\alpha|=k} \frac{k!}{\alpha!} w^\alpha x^\alpha \quad \Longrightarrow \quad 0 = \mathfrak{q}(p_w) = k! \sum_{|\alpha|=k} q_\alpha w^\alpha \quad \forall w \in \mathbb{R}^d \quad \Longrightarrow \quad \mathfrak{q} = 0.$$

Contradiction. ∎

Step 2: the smooth case

Lemma 7.2.4
If $\sigma \in C^\infty(\mathbb{R})$ is not a polynomial, $\mathcal{X} := \mathcal{N}_1^1(\sigma; 1)$ is a universal approximator of $C^0(\mathbb{R})$.

Proof. It suffices (Stone–Weierstrass) to show that $x^k \in \overline{\mathcal{X}}$ for every k. Set $f_b(x, w) := \sigma(wx + b)$.

- $\partial_w^k f_b(x, w) = x^k \sigma^{(k)}(wx + b)$, hence $\partial_w^k f_b(x, 0) = x^k\, \sigma^{(k)}(b)$. Since σ is not a polynomial, for every k there is b_k with $\sigma^{(k)}(b_k) \neq 0$. It therefore suffices to show $\partial_w^k f_b(\cdot, 0) \in \overline{\mathcal{X}}$.
- Finite difference: the two-neuron network
$$\phi_\theta(x) = \frac{1}{h}\sigma((w + h)x + b) - \frac{1}{h}\sigma(wx + b) \qquad \left(W_1 = \begin{pmatrix} w+h \\ w \end{pmatrix},\ b_1 = \begin{pmatrix} b \\ b \end{pmatrix},\ W_2 = \begin{pmatrix} \frac{1}{h} & -\frac{1}{h} \end{pmatrix},\ b_2 = 0\right)$$
satisfies, by Taylor, $\phi_\theta(x) = \partial_w f_b(x, w) + \frac{h}{2} x^2 \sigma''(\xi x + b)$, $\xi \in [w, w + h]$.
- On a compact K, $\sup_{x \in K} \sup_{|h|<1} \left|x^2 \sigma''(\xi x + b)\right| < \infty$, hence
$$\phi_\theta \to \partial_w f_b(\cdot, w) \quad \text{uniformly on } K \text{ as } h \to 0.$$
- By induction, $\partial_w^k f_b(\cdot, w) \in \overline{\mathcal{X}}$. ∎

Step 3: regularization by convolution

Definition 7.2.3 (Convolution) and Lemma 7.2.5

Let σ be bounded and piecewise continuous. For every $\varphi \in C_c^\infty(\mathbb{R})$,

$$\sigma \star \varphi \in \overline{\mathcal{N}_1^1(\sigma; 1)}, \qquad \sigma \star \varphi(x) := \int_{\mathbb{R}} \sigma(x - y)\varphi(y)\,\mathrm{d}y \in C^\infty(\mathbb{R}).$$

Moreover, if $\sigma \star \varphi$ is a polynomial for every φ, then σ is a polynomial [Petersen, Zech 2025, Lemma 3.15].

Idea of the proof. Riemann sum: with $\operatorname{supp} \varphi \subset [-a, a]$, $h_n = 2a/n$, $y_j = -a + jh_n$,

$$f_n(x) := h_n \sum_{j=0}^{n-1} \sigma(x - y_j)\,\varphi(y_j)$$

is a network with n neurons (weights 1, biases $-y_j$, output weights $h_n\varphi(y_j)$).
One shows $f_n \to \sigma \star \varphi$ uniformly on compact sets, by isolating the discontinuity points of σ. ■

Conclusion of Theorem 7.2.1. If σ is not a polynomial, there is φ such that $\tilde{\sigma} = \sigma \star \varphi$ is smooth and not a polynomial.
By step 2, $\mathcal{N}_1^1(\tilde{\sigma}; 1)$ is universal. By step 3, $\mathcal{N}_1^1(\tilde{\sigma}; 1) \subset \overline{\mathcal{N}_1^1(\sigma; 1)}$. By step 1, we pass to $\mathbb{R}^d$. ■

Universal approximation for deep networks

Corollary 7.2.6 (Universal approximation for deep neural networks)

Let σ be bounded and piecewise continuous. For every $L \in \mathbb{N}$, $\mathcal{N}_d^1(\sigma; L)$ is a universal approximator of $C^0(\mathbb{R}^d)$ if and only if σ is not a polynomial.

Everything rests on the possibility of approximating the identity with additional hidden layers.

Proposition 7.2.1 Exercise

Let σ be differentiable at a point $s^\star$ with $\sigma'(s^\star) \neq 0$. For every L, every compact $K \subset \mathbb{R}^d$ and every $\varepsilon > 0$, there is $\phi \in \mathcal{N}_d^d(\sigma; L, d)$ such that $\left|\phi(x) - x\right| \leq \varepsilon$ for every $x \in K$.

- **Idea** ($L = 1$): with $b = (s^\star, \dots, s^\star)$, $\varsigma = \sigma'(s^\star)$ and $\lambda \gg 1$,
$$\phi_\lambda(x) := \frac{\lambda}{\varsigma}\,\sigma\Big(\frac{x}{\lambda} + b\Big) - \frac{\lambda}{\varsigma}\,\sigma(b) = x + o(1) \quad (\lambda \to \infty).$$
We zoom in on a point where σ is locally linear.
- **Proof of the corollary**: take $\phi_1 \in \mathcal{N}_d^1(\sigma; 1)$ with $\left\|f - \phi_1\right\|_K \leq \varepsilon/2$. On a compact neighborhood K' of K, ϕ_1 is uniformly continuous: $\left|\phi_1(z) - \phi_1(x)\right| \leq \varepsilon/2$ when $|z - x| \leq \rho$. Apply first a depth-$(L-1)$ network ρ-close to the identity on K (Prop. 7.2.1), then ϕ_1: a network in $\mathcal{N}_d^1(\sigma; L)$, ε-close to f on K. ■

Going further: superexpressive activations

Theorem 7.2.7 (Universal approximation via superexpressive activations) [Petersen, Zech 2025, Cor. 3.23]
There is a *continuous* activation $\sigma : \mathbb{R} \to \mathbb{R}$ such that for every compact $K \subset \mathbb{R}^d$, every $\varepsilon > 0$ and every $f \in C(K)$, there is $\phi \in \mathcal{N}_d^1(\sigma; 2, 2d^2 + d)$ with $\left|f(x) - \phi(x)\right| \leq \varepsilon$ for every $x \in K$.

- Depth 2 and width $2d^2 + d$: the size is independent of f, of K and of ε!
- It rests on the Kolmogorov superposition theorem (1957): every $f \in C([0,1]^d)$ can be written as $f(x) = \sum_{q=0}^{2d} \Phi_q\left(\sum_{p=1}^{d} \psi_{pq}(x_p)\right)$ with continuous functions *of one variable*.
- But the activation is pathological (it encodes all polynomials with rational coefficients): no practical interest. It shows that counting parameters is a delicate criterion.

What universal approximation does not say

- Rate: how many neurons to reach ε? For $f \in C^s$, typically $n \sim \varepsilon^{-d/s}$ (curse of dimensionality).
- Role of depth: some functions need exponentially more neurons at depth 1 than at depth L [Telgarsky 2016].
- Nothing about the algorithm that finds θ: that was Section 7.1.

Summary: universal approximation

What we proved

- A network with one hidden layer approximates every continuous function on every compact set, as soon as σ is not a polynomial (Theorem 7.2.1).
- The proof goes through three reductions: dimension one (Stone–Weierstrass and Hahn–Banach), the smooth case (finite differences of $\sigma(wx + b)$), regularization (convolution).
- Deep networks inherit the property by approximating the identity layer by layer (Corollary 7.2.6).

What we did not prove

- How many neurons are needed: the curse of dimensionality is not lifted by the theorem.
- Why depth helps in practice.
- How to find the parameters θ efficiently when the network is deep.

Universal approximation guarantees that the family $\mathcal{F}$ is not the obstacle. The obstacles are optimization (Section 7.1: vanishing gradients) and depth: that is chapter 8.

Chapter 7 in one slide

Take-away

- A feed-forward network is a composition of affine maps and a componentwise nonlinearity σ. Without σ, nothing beyond affine maps (XOR).
- Learning is gradient descent on the empirical risk. Backpropagation is the chain rule organized from the output to the input: one forward pass, one backward pass, cost $\approx 2\times$ one evaluation.
- The multiplicative structure of the chain rule produces vanishing gradients and dead neurons in deep networks.
- Universal approximation: one hidden layer suffices as soon as σ is not a polynomial (Stone–Weierstrass). Deep networks inherit it by approximating the identity.

result	statement	hypothesis on σ
Theorem 7.1.1	$\partial\mathcal{L}/\partial W_\ell = \alpha^{(\ell)}\sigma(\tilde{x}^{(\ell-1)})^\top$, adjoints by backward recursion	differentiable
Theorem 7.2.1	$\mathcal{N}_d^1(\sigma;1)$ universal $\iff$ σ not a polynomial	bounded, piecewise continuous
Corollary 7.2.6	same for $\mathcal{N}_d^1(\sigma;L)$, every L	idem, $\sigma'(s^\star)\neq 0$ at some $s^\star$
Theorem 7.2.7	depth 2, width $2d^2+d$ suffice for everything	one pathological continuous σ

8

chapter

Residual neural networks, neural ODEs and control

- Control formalism via neural ODEs

The idea of residual networks [He et al. 2016]

Diagnosis [Scagliotti 2023]. For $\phi_\theta = \phi^L_{\theta_L} \circ \ldots \circ \phi^1_{\theta_1}$,

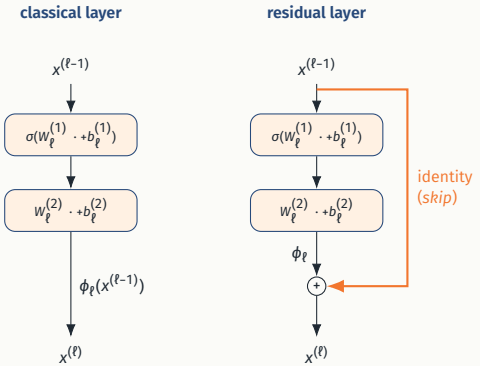
$$\frac{\partial}{\partial \theta_1}\mathcal{L} = \mathrm{d}_x \mathcal{L} \cdot D_x \phi^L_{\theta_L} \cdots D_x \phi^2_{\theta_2} \cdot D_{\theta_1} \phi^1_{\theta_1}.$$

At initialization, $|\theta_i| \ll 1$ and often $\theta_i = 0 \Rightarrow \phi^i_{\theta_i} \equiv 0$, so that $\left|D_x \phi^i_{\theta_i}\right| \ll 1$ and the product collapses.

Idea (2015): design the layers so that $|\theta_i| \approx 0 \Rightarrow D_x \phi^i_{\theta_i} \approx \mathrm{Id}$.

The solution: learn the residual,

$$\phi^i_{\theta_i}(x) = x + F(x, \theta_i), \qquad F(\cdot, 0) \equiv 0.$$



Residual neural networks (ResNets)

Definition 8.0.1 (Residual neural network)

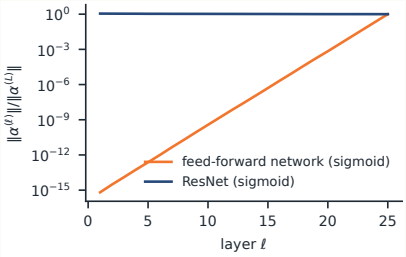
Let σ be an activation function and $d \in \mathbb{N}$. An L-hidden layer ResNet $\Phi_\theta : \mathbb{R}^d \to \mathbb{R}^d$ is given by the parameters $\theta = (W_\ell^{(1)}, b_\ell^{(1)}, W_\ell^{(2)}, b_\ell^{(2)})_{\ell=1}^L$, with $W_\ell^{(i)} \in \mathbb{R}^{d\times d}$ and $b_\ell^{(i)} \in \mathbb{R}^d$, and

$$\Phi_\theta = \Phi_L \circ \ldots \circ \Phi_1, \qquad \Phi_\ell(x) = x + \phi_\ell(x), \qquad \phi_\ell(x) = W_\ell^{(2)}\,\sigma(W_\ell^{(1)}x + b_\ell^{(1)}) + b_\ell^{(2)}.$$

- The notes fix the hidden width to d. With $W_\ell^{(1)} \in \mathbb{R}^{d_1\times d}$, $W_\ell^{(2)} \in \mathbb{R}^{d\times d_1}$ it is d_1, the *width* of Section 8.1.
- Input and output of the same dimension d: layers $\phi^{\text{pre}} : \mathbb{R}^{d_{\text{in}}} \to \mathbb{R}^d$ and $\phi^{\text{post}} : \mathbb{R}^d \to \mathbb{R}^{d_{\text{out}}}$ are added when needed.
- Backpropagation becomes additive (equation (8.5) of the notes):

$$\alpha^{(\ell)} = \alpha^{(\ell+1)} + \left(\frac{\partial \phi_{\ell+1}}{\partial x^{(\ell)}}\right)^\top \alpha^{(\ell+1)}.$$

  $\rightsquigarrow$ the gradient propagates without dying out.



Norms of the adjoints at initialization ($d = 50$, Gaussian weights).

Close to the identity: a lemma

Residual layer $\Phi_\ell = \mathrm{Id} + \phi_\ell$, adjoint $\alpha^{(\ell)} = (I + D\phi_{\ell+1}(x^{(\ell)}))^\top \alpha^{(\ell+1)}$. What happens when ϕ_ℓ is small?

Lemma

Let $\phi : \mathbb{R}^d \to \mathbb{R}^d$ be of class C^1 with $\|D\phi(x)\| \le q < 1$ for every x. Then:

1. for every x, $I + D\phi(x)$ is invertible, with $\|I + D\phi(x)\| \le 1 + q$ and $\|(I + D\phi(x))^{-1}\| \le \frac{1}{1-q}$.
2. $\Phi = \mathrm{Id} + \phi$ is a C^1-diffeomorphism of $\mathbb{R}^d$, and Φ^{-1} is $\frac{1}{1-q}$-Lipschitz.

Proof.

- (1) With $A = D\phi(x)$: $\|I + A\| \le 1 + q$, and $(I + A)^{-1} = \sum_{k\ge 0}(-A)^k$ (Neumann series) converges, with norm $\le \sum_k q^k = \frac{1}{1-q}$.
- (2) *Injective*: if $\Phi(x) = \Phi(y)$, then $\|x - y\| = \|\phi(y) - \phi(x)\| \le q\,\|x - y\|$, hence $x = y$.
- (2) *Surjective*: for fixed y, $x \mapsto y - \phi(x)$ is a q-contraction. Its fixed point (Banach) satisfies $\Phi(x) = y$.
- (2) *Inverse*: $\|x - x'\| \le \|\Phi(x) - \Phi(x')\| + q\,\|x - x'\|$ gives the Lipschitz constant, and Φ^{-1} is C^1 by (1). ■

A layer close to the identity does not fold the data, and changes the norm of the adjoint only by a factor between $1 - q$ and $1 + q$.

Consequence: the gradients of a ResNet do not vanish

Recall: $a^{(\ell)} = \partial \mathcal{L} / \partial x^{(\ell)}$ and $\partial \mathcal{L} / \partial \theta_\ell = (\partial \phi_\ell / \partial \theta_\ell)^\top a^{(\ell)}$. If $a^{(\ell)}$ dies out, layer ℓ stops learning.

Take L layers with $\|D\phi_\ell\| \leq c/L$: the scale of an Euler step (Section 8.1), or of a small initialization.

Corollary
Under this assumption, $\Phi_\theta = \Phi_L \circ \ldots \circ \Phi_1$ is a C^1-diffeomorphism of $\mathbb{R}^d$ and, for every $\ell \in [\![0, L]\!]$,

$$(1 - c/L)^L \left\|a^{(L)}\right\| \leq \left\|a^{(\ell)}\right\| \leq (1 + c/L)^L \left\|a^{(L)}\right\| \leq e^c \left\|a^{(L)}\right\|.$$

Proof. Point (1) of the lemma with $q = c/L$ and $\left\|A^\top\right\| = \|A\|$:

$$(1 - c/L) \left\|a^{(\ell+1)}\right\| \leq \left\|a^{(\ell)}\right\| \leq (1 + c/L) \left\|a^{(\ell+1)}\right\|, \qquad \text{applied } L - \ell \text{ times.}$$

Each Φ_ℓ is a diffeomorphism by point (2), hence so is their composition. ■

- Bounds independent of L, with $(1 - c/L)^L \to e^{-c}$: every layer receives a gradient of the same order as $a^{(L)}$.
- Classical network at the same scale, $x^{(\ell)} = \phi_\ell(x^{(\ell-1)})$: $\left\|a^{(\ell)}\right\| \leq (c/L)^{L-\ell} \left\|a^{(L)}\right\|$, the gradient vanishes geometrically.

A ResNet transports the data without crushing them, and the gradient of the loss reaches every layer, even the first ones.

Why residual networks matter

- **Depth.** ResNet (He, Zhang, Ren, Sun 2016): 152 layers, winner of ImageNet 2015.
  Without residual connections, a 56-layer network has a larger *training* error than a 20-layer one (CIFAR-10). This is not overfitting, it is the optimization that fails.
- **Everywhere.** The residual block is the building brick of almost every current architecture. Each layer of a transformer (attention, then MLP) is of the form $x \mapsto x + \phi(x)$, and so are the U-Nets of diffusion models.
- **For us.** $x^{(\ell)} = x^{(\ell-1)} + \phi_\ell(x^{(\ell-1)})$ reads as an Euler step. This opens the door to neural ODEs and to control theory (Section 8.1).

A small architectural change, $x + \phi(x)$ instead of $\phi(x)$, makes training possible at depth 10^2 to 10^3 and gives the network a dynamical structure.

8.1

Residual neural networks, neural ODEs and control

Control formalism via neural ODEs

From the ResNet to an ODE [Haber, Ruthotto 2017; E 2017; Chen et al. 2018]

Write $u = (W^{(1)}, b^{(1)}, W^{(2)}, b^{(2)})$ and

$$F(x, u) := W^{(2)}\sigma(W^{(1)}x + b^{(1)}) + b^{(2)}.$$

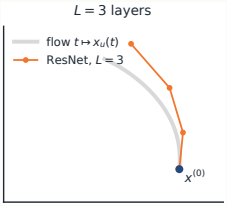
With a step $\eta > 0$, a layer of the ResNet reads

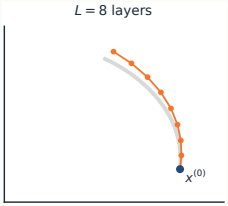
$$x^{(\ell)} = x^{(\ell-1)} + \eta\, F(x^{(\ell-1)}, u_\ell) \quad \Longleftrightarrow \quad \frac{x^{(\ell)} - x^{(\ell-1)}}{\eta} = F(x^{(\ell-1)}, u_\ell), \qquad \ell \in [\![1, L]\!].$$

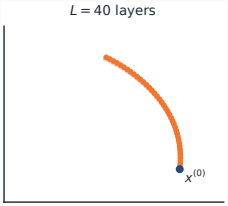
This is the explicit Euler scheme for the *controlled* differential equation

$$\dot{x}(t) = F(x(t), u(t)), \quad t \in [0, L\eta], \qquad u(t) = u_\ell \text{ for } t \in [(\ell-1)\eta, \ell\eta).$$

With $\eta = 1/L$ (total time 1) and $L \to \infty$: continuous-time limit on $[0, 1]$.







Definition 8.1.1 (Neural ODEs control systems)

Let σ be an activation function and $F : \mathbb{R}^d \times U \to \mathbb{R}^d$, $F(x,u) = W^{(2)}\sigma(W^{(1)}x + b^{(1)}) + b^{(2)}$ for $u = (W^{(1)}, b^{(1)}, W^{(2)}, b^{(2)}) \in U \subset \mathbb{R}^{d_1 \times d} \times \mathbb{R}^{d_1} \times \mathbb{R}^{d \times d_1} \times \mathbb{R}^d$. A neural ODE is the controlled ODE

$$\dot{x}(t) = F(x(t), u(t)), \qquad t \in [0,1],$$

with initial condition $x(0) = x \in \mathbb{R}^d$ and control $u \in L^\infty([0,1]; U)$. Its flow is the map $\Phi_u : \mathbb{R}^d \to \mathbb{R}^d$, $\Phi_u(x) := x_u(1)$, where x_u is the solution with control u and initial condition x.

ResNet

$$x^{(\ell)} = x^{(\ell-1)} + \eta\, F(x^{(\ell-1)}, u_\ell)$$

$\longleftrightarrow$

Neural ODE

$$\dot{x}(t) = F(x(t), u(t))$$

- Each layer is an Euler step of length $1/L$: the output $x^{(L)}$ approximates $x_u(1)$.
- The weights u_ℓ play the role of the control $u(t)$ on $[(\ell-1)/L, \ell/L)$.
- "Very deep" ResNets are therefore fine discretizations of one and the same continuous object.

The ResNet as an Euler scheme of a control system

Theorem 8.1.1 (Neural ODEs)

Let $u : [0,1] \to U$ be piecewise continuous and bounded. For every $\varepsilon > 0$ there exists $L_0 \in \mathbb{N}$ such that for every $L \geq L_0$ the solution x_u of the neural ODE satisfies

$$\left|x_u(\ell/L) - x^{(\ell)}\right| \leq \varepsilon, \quad \forall \ell \in [\![1, L]\!], \qquad \text{where} \quad x^{(\ell)} = x^{(\ell-1)} + \frac{1}{L} F\left(x^{(\ell-1)}, u(\tfrac{\ell-1}{L})\right).$$

- This is the convergence of the Euler scheme (Grönwall lemma, F locally Lipschitz in x).
- A ResNet with $L \gg 1$ layers is therefore an approximation of the flow: $\Phi_u \approx \Phi_L \circ \ldots \circ \Phi_1$.
- Other choices of F (other layer architectures) are of course possible.

The neural ODE formalism lets us use the theory of control to study and train deep networks:

- the expressivity of the ResNet is a controllability property of the neural ODE,
- the training of a ResNet with L layers is an optimal control problem for the neural ODE, with piecewise constant controls with L pieces.

The ResNet / control dictionary [Scagliotti 2023]

ResNet		Control system
layers $\ell = 1, \dots, L$	$\leftrightarrow$	time $t \in [0, 1]$
$\Phi_\ell(x) = x + \frac{1}{L} F(x, u_\ell)$	$\leftrightarrow$	$\dot{x}(t) = F(x(t), u(t))$
weights $u_\ell \in \mathbb{R}^m$	$\leftrightarrow$	control $u \in L^\infty([0, 1]; \mathbb{R}^m)$
composition $\Phi_L \circ \dots \circ \Phi_1$	$\leftrightarrow$	flow $\Phi_u = \Phi_u^1$
expressivity of the network	$\leftrightarrow$	controllability of the system
training of the network	$\leftrightarrow$	optimal control
backpropagation	$\leftrightarrow$	adjoint equation (Pontryagin maximum principle)

The whole theory of control can be mobilized to study and train deep networks.

$\rightsquigarrow$ the system is nonlinear: the linear tools of chapters 4 and 5 (Kalman, Riccati) do not apply directly.

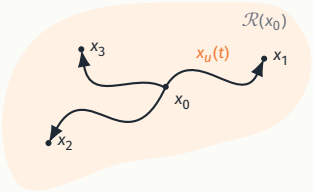
Recall from chapter 4: what is control theory?

A control system on $\mathbb{R}^d$ (Definition 4.1.1):

$$\dot{x} = F(x, u), \quad x \in \mathbb{R}^d,\ u(t) \in U \subset \mathbb{R}^m$$

- $u(\cdot)$: the *control*, chosen by the user.
- $x_u(\cdot)$: the associated *trajectory*, $\Phi_u^t(x_0) = x_u(t)$ its flow.
- *Reachable set* from x_0:

$$\mathcal{R}(x_0) = \{\Phi_u^t(x_0) \mid t \geq 0,\ u\}.$$



The two main questions of control theory

1. Controllability: can x_0 be steered to x_1 (that is, $\mathcal{R}(x_0) = \mathbb{R}^d$)? Under which constraints?
2. Optimal control: among the controls that do it, which one minimizes a cost $J(u)$?

Example. The Reeds–Shepp car: states $(x, y, \vartheta) \in \mathbb{R}^2 \times \mathbb{S}^1$, controls $(u_1, u_2) \in \mathbb{R}^2$,

$$\begin{cases} \dot{x} = u_1 \cos\vartheta \\ \dot{y} = u_1 \sin\vartheta \\ \dot{\vartheta} = u_2 \end{cases}$$

A nonlinear system with two controls in dimension three, and yet controllable.
(One can parallel park!)

8.1.1 Training via optimal control

Learning = optimal control

Data $\mathfrak{X} = \{(x_i, y_i)\}_{i=1}^N \subset \mathbb{R}^d \times \mathbb{R}^d$, with pairwise distinct x_i. Training a ResNet with L layers amounts to the optimal control problem

$$\text{find } u^\star = \arg\min_{u \in L^\infty([0,1];U)} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(\Phi_u(x_i), y_i), \tag{8.17}$$

with u piecewise constant (L pieces).

Under the mild assumption $\mathcal{L}(y, y) = 0$ for every $y \in \mathbb{R}^d$, the minimum of (8.17) is zero as soon as a single control u steers every x_i to its target y_i. This is a controllability property of the neural ODE, stronger than the controllability of Definition 4.1.3 (next frame).

If such a control exists, the network memorizes the data: zero training error.

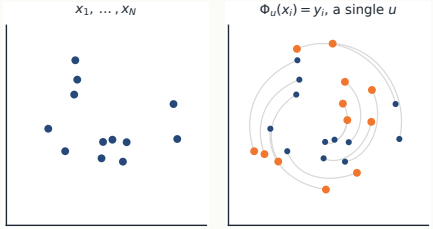
Simultaneous controllability

Definition 8.1.2 (Simultaneous controllability)

The neural ODE is simultaneously controllable on a set $\Omega_N \subset (\mathbb{R}^d)^N$ of admissible ensembles if for every $N \in \mathbb{N}$, every ensemble $(x_1, \dots, x_N) \in \Omega_N$ and every ensemble of targets $(y_1, \dots, y_N) \in \Omega_N$ there exists $u \in L^\infty([0,1]; U)$ such that

$$\Phi_u(x_i) = y_i, \qquad \forall i \in [\![1, N]\!].$$

The same control u must steer every x_i to its y_i: this is much stronger than the controllability of Definition 4.1.3.



The same u for every point. The larger N, the harder.

If the neural ODE is simultaneously controllable and $\mathcal{L}(y, y) = 0$, then (8.17) is solved at zero cost.

The activations of Tabuada and Gharesifard

The simplest ResNet of Definition 8.1.1: hidden width $d_1 = d$, $W_\ell^{(2)} = s_\ell$ Id scalar and $b_\ell^{(2)} = 0$,

$$x^{(\ell)} = x^{(\ell-1)} + \eta\, s_\ell\, \Sigma\big(W_\ell x^{(\ell-1)} + b_\ell\big) \quad \longleftrightarrow \quad \dot{x}(t) = s(t)\,\Sigma\big(W(t)x(t) + b(t)\big),$$

with control $u = (s, W, b) \in \mathbb{R} \times \mathbb{R}^{d\times d} \times \mathbb{R}^d$ and $\Sigma(x) = (\sigma(x_1), \dots, \sigma(x_d))$.

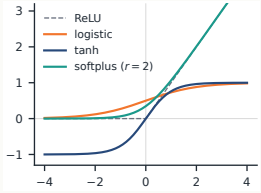
Assumption on the activation (Theorems 8.1.2 to 8.1.4)

σ is Lipschitz with $\sigma' \geq 0$, and there exist $j \in \mathbb{N}_0$ and $a_1, a_2, a_3 \in \mathbb{R}$ with $a_3 \neq 0$ such that $\eta := \sigma^{(j)}$ is injective and solves the quadratic ODE

$$\eta' = a_1 + a_2\eta + a_3\eta^2.$$

activation	definition	equation
logistic (sigmoid)	$\sigma(x) = 1/(1+e^{-x})$	$\sigma' = \sigma - \sigma^2$
hyperbolic tangent	$\sigma(x) = \tanh x$	$\sigma' = 1 - \sigma^2$
softplus	$\sigma(x) = \frac{1}{r}\log(1+e^{rx})$	$\sigma'' = r\sigma' - r(\sigma')^2$

ReLU is the limit $r \to \infty$ of softplus. All are *analytic*.



Simultaneous controllability for generic inputs

Theorem 8.1.2 (Simultaneous controllability for generic inputs) [Tabuada, Gharesifard 2023]

Under the assumption on σ of the previous slide, the neural ODE of Definition 8.1.1 is simultaneously controllable on the set of ensembles whose points are pairwise distinct in every coordinate,

$$\Omega_N = \left\{(x_1, \dots, x_N) \in (\mathbb{R}^d)^N \;\middle|\; (x_i)_k \neq (x_j)_k,\ \forall i \neq j,\ \forall k \in [\![1, d]\!]\right\}.$$

- The result already holds with the restricted controls (s, W, b) of the previous slide, hence for the neural ODE of Definition 8.1.1.
- Ω_N is open and dense in $(\mathbb{R}^d)^N$: generic data can be memorized exactly, any data up to ε.
- The memorization capacity is independent of N: the depth (the time) adapts.

Proved with the Chow–Rashevskii theorem of geometric control: the ensemble system on $(\mathbb{R}^d)^N$ is driven by a family of vector fields whose iterated Lie brackets span the whole tangent space, which is where the quadratic ODE for η is used. Outside the scope of this course. The same theorem is the first step of the universal approximation results of Section 8.1.2.

Two problems with exact memorization, and a way out

Two problems with this approach (the notes, Section 8.1.1).

- The controllability result is not constructive: it does not say how to compute the control u steering the x_i to the y_i.
- Striving for exactly zero training error leads to overfitting, hence poor generalization on unseen data.

Alternative [Scagliotti 2023]: relax the problem and minimize the regularized cost

$$J(u) = \frac{1}{N}\sum_{i=1}^{N} \left\| \Phi_u(x_i) - y_i \right\|_2^2 + \frac{\beta}{2}\int_0^1 \left\| u(t) \right\|_2^2 \, \mathrm{d}t. \tag{8.21}$$

- (8.21) is an optimal control problem of the type of chapter 5, with a quadratic penalty on the control, but the dynamics is nonlinear: no Riccati equation.
- Its gradient is computed by an adjoint equation, integrated backward from the output: the continuous-time version of backpropagation (Theorem 7.1.1), and the Pontryagin maximum principle of chapter 5. Not pursued here.

8.1.2 Expressivity via controllability

Expressivity = approximation of diffeomorphisms

Proposition 8.1.1

Let $\mathrm{ResNet}_d(\sigma; m)$ denote the set of ResNets with activation σ, input/output dimension d and width at most m. Then the closure of $\mathrm{ResNet}_d(\sigma; d)$ for the uniform convergence on compact sets contains

$$\{\Phi_u \mid u \in L^\infty([0,1]; U)\}.$$

In particular, if $\{\Phi_u\}_u$ is dense in $C^0(K, \mathbb{R}^d)$ for every compact K, then $\mathrm{ResNet}_d(\sigma; d)$ is a universal approximator of $C^0(\mathbb{R}^d, \mathbb{R}^d)$.

Every flow Φ_u is a diffeomorphism of $\mathbb{R}^d$. Expressivity of the ResNet = controllability in the space of diffeomorphisms:

$$\exists\, \theta \text{ s.t. } f \simeq \Phi_\theta \iff \exists\, u \text{ s.t. } f \simeq \Phi_u.$$

- Beware: density of $\{\Phi_u\}_u$ in $C^0(K, \mathbb{R}^d)$ is much stronger than (simultaneous) controllability: for a given continuous $f : K \to \mathbb{R}^d$, *one* u must send *every* $x \in K$ to $f(x)$.
- **Obstruction.** Φ_u is injective, f need not be. The way out: embed into a larger dimension (Theorem 8.1.4).

Functions homotopic to the identity

Definition 8.1.3 (Monotone analytic homotopy)

A function $f : \mathbb{R}^d \to \mathbb{R}^d$ is monotone homotopic to the identity if there exists an analytic homotopy $h : [0,1] \times \mathbb{R}^d \to \mathbb{R}^d$, that is, h analytic with $h(0,x) = x$ and $h(1,x) = f(x)$, such that for every $t \in [0,1]$ the map $h(t,\cdot)$ is monotone:

$$x_i \le y_i \quad \forall i \in [\![1,d]\!] \quad \Longrightarrow \quad h(t,x)_i \le h(t,y)_i \quad \forall i \in [\![1,d]\!].$$

- Monotone: the partial order $x \preceq y$ ($x_i \le y_i$ for every i) is preserved by $h(t,\cdot)$. For $h(t,\cdot) \in C^1$, this means $\partial h(t,\cdot)_i / \partial x_j \ge 0$.
- The flows of the neural ODE with $\sigma' \ge 0$ and diagonal W are monotone [Tabuada, Gharesifard 2023, Prop. 5.4], which is why monotonicity enters the statement.
- Monotonicity controls what happens between the sample points, which is what Theorem 8.1.2 says nothing about.

Universal approximation of functions homotopic to the identity

Theorem 8.1.3 (Universal approximation of functions homotopic to the identity) [Tabuada, Gharesifard 2023]

Assume that σ satisfies the assumption of Theorem 8.1.2. Then for every compact $K \subset \mathbb{R}^d$, every continuous $f : K \to \mathbb{R}^d$ monotone analytic homotopic to the identity and every $\varepsilon > 0$ there exists $u \in L^\infty([0,1]; U)$ whose flow satisfies

$$\left\| f - \Phi_u \right\|_K \leq \varepsilon.$$

In particular, $\mathrm{ResNet}_d(\sigma; d)$ is a universal approximator of the functions that are monotone analytic homotopic to the identity on compact sets.

- Stated without proof (the idea is on the last frame of this section).
- The restriction to functions homotopic to the identity is natural: every flow Φ_u is itself homotopic to the identity through $t \mapsto \Phi_u^t$.

Universal approximation of ResNets

Theorem 8.1.4 [Tabuada, Gharesifard 2023]

Assume that σ satisfies the assumption of Theorem 8.1.2 and let d' be such that $d = 2d' + 1$. Then for every compact $K \subset \mathbb{R}^{d'}$, every continuous $f : K \to \mathbb{R}^{d'}$ and every $\varepsilon > 0$ there exist a control $u \in L^\infty([0,1]; U)$, an injection $\alpha : \mathbb{R}^{d'} \to \mathbb{R}^{2d'+1}$ and a projection $\beta : \mathbb{R}^{2d'+1} \to \mathbb{R}^{d'}$ such that

$$\left\| f - \beta \circ \Phi_u \circ \alpha \right\|_K \leq \varepsilon.$$

In particular, the set $\mathrm{ResNet}_{d'}(\sigma; 2d'+1)$ of ResNets with input/output dimension d' and width at most $2d' + 1$ is a universal approximator of $C^0(\mathbb{R}^{d'}, \mathbb{R}^{d'})$.

- The embedding α and the projection β bypass the obstruction: Φ_u is injective on $\mathbb{R}^{2d'+1}$, while f need not be.
- In the network, α and β are the layers ϕ^{pre} and ϕ^{post} added to the ResNet of Definition 8.0.1, with $d_{\mathrm{in}} = d_{\mathrm{out}} = d'$.

Fixed width $2d' + 1$ and arbitrary depth: the exact dual of shallow networks (Theorem 7.2.1), where the depth is fixed and the width is free.

How the three theorems fit together (proofs not covered)

1. **Theorem 8.1.2, memorization of a finite sample.** Controllability of the ensemble system on $(\mathbb{R}^d)^N$ minus the ensembles with a repeated coordinate (the *folded space*), by Chow–Rashevskii.
2. **Theorem 8.1.3, from finite samples to compact sets.** Sample K on a fine grid, follow the homotopy h with a monotone flow (analyticity: the order of the coordinates changes finitely many times along the way). Between two grid points a monotone map is trapped between its values at those points, which gives the uniform estimate.
3. **Theorem 8.1.4, general continuous functions.** Stone–Weierstrass (approximate f by a polynomial) and an embedding into $\mathbb{R}^{2d'+1}$ that makes the homotopy to the identity monotone.

Chow–Rashevskii (memorization) → monotonicity (compact sets) → Stone–Weierstrass + embedding (continuous functions)

The tools (Lie brackets, Chow–Rashevskii theorem) belong to geometric control theory. See the paper of Tabuada and Gharesifard, and the references in the notes.

Chapter 8 in one slide

Take-away

- A residual layer $x \mapsto x + \phi(x)$ is close to the identity: it does not fold the data and lets the gradient reach every layer (lemma and corollary).
- A ResNet is the explicit Euler scheme of the neural ODE $\dot{x} = F(x, u)$, and approximates its flow Φ_u (Theorem 8.1.1).
- Training is an optimal control problem (8.17). Simultaneous controllability (Theorem 8.1.2) gives zero training error, which is neither constructive nor desirable: the regularized cost (8.21) is the better problem.
- Expressivity is controllability in the space of diffeomorphisms (Proposition 8.1.1). With width $2d' + 1$ and free depth, ResNets are universal approximators (Theorem 8.1.4).

Dictionary: layers ↔ time, weights ↔ control, composition ↔ flow, expressivity ↔ controllability, training ↔ optimal control, backpropagation ↔ adjoint equation.

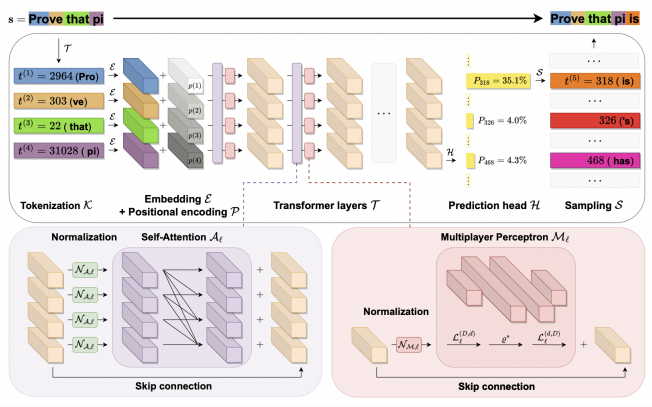
9

epilogue

Epilogue: large language models and transformers

A language model, end to end [Frieder et al. 2024]

An autoregressive LLM predicts the next piece of a word (*token*) from the previous ones.

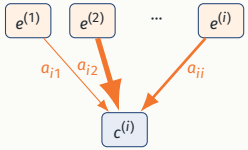


Operations of GPT-2, from [Frieder et al. 2024]: tokenization, embedding, L transformer blocks, prediction head, sampling.

Why attention? [Welleck 2025]

- Toy problem: predict the next letter of a first name, with letters embedded as $e^{(j)} = \mathcal{E}(t_j) \in \mathbb{R}^d$ ($\mathcal{E}$: a learned map from the alphabet to $\mathbb{R}^d$).
- The prediction head receives a single vector $c^{(i)}$, the context, a summary of $e^{(1)}, \dots, e^{(i)}$. How to build it?

1. **Bigram**: $c^{(i)} = e^{(i)}$. Only the last letter matters.
2. **Average**: $c^{(i)} = \frac{1}{i} \sum_{j \le i} e^{(j)}$. Every letter weighs the same, the order is lost.
3. **Learned weighted average**: $c^{(i)} = \sum_{j \le i} a_{ij}\, v^{(j)}$, with weights a_{ij} depending on the letters and vectors $v^{(j)}$ computed from $e^{(j)}$ (next frame).



attention: learned weights

Attention is a weighted average of the past letters, with learned weights. How are the weights a_{ij} computed?

Attention: queries, keys and values [Welleck 2025]

Transformer. Each token emits three vectors, through three learned linear maps:

- the query $q^{(i)} = Qe^{(i)}$: "what I am looking for".
- the key $k^{(j)} = Ke^{(j)}$: "what I contain".
- the value $v^{(j)} = Ve^{(j)}$: "what I pass on when someone looks at me".

Weights: $a_{ij} = \mathrm{softmax}_{j \leq i}\left(\left\langle q^{(i)}, k^{(j)} \right\rangle / \sqrt{d_k}\right) = \exp\left(\left\langle q^{(i)}, k^{(j)} \right\rangle / \sqrt{d_k}\right) / \sum_{l \leq i} \exp\left(\left\langle q^{(i)}, k^{(l)} \right\rangle / \sqrt{d_k}\right)$, with d_k the dimension of the keys. A query aligned with a key gives a large weight.

The context is then $c^{(i)} = \sum_{j \leq i} a_{ij}\, v^{(j)}$: every token looks at the previous ones, and decides how much to take from each one.

Attention is the only operation that mixes tokens. It will be read below as an interaction between particles.

Training: predict the next token [Welleck 2025]

- **Data.** Sequences of tokens $t_1 \dots t_n$: texts, or strings produced by a program. No label to provide: at position i, the target is t_{i+1}, the token that follows.
- **Causal mask.** In the attention, position i only looks at positions $j \leq i$. The output at position i is therefore a probability distribution $p_\theta(\cdot \mid t_1, \dots, t_i)$ on the vocabulary $\mathcal{T}$ (the set of tokens), and a single forward pass on $t_1 \dots t_n$ gives the n predictions at once.
- **Loss.** Cross-entropy, summed over the positions and over the sequences of the data set:

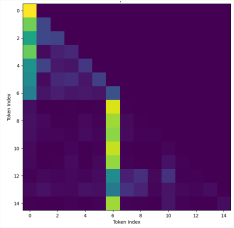
$$\mathcal{L}(\theta) = - \sum_{\text{sequences}} \sum_{i=1}^{n-1} \log p_\theta(t_{i+1} \mid t_1, \dots, t_i).$$

- **Optimization.** Stochastic gradient descent and backpropagation, the controls being Q, K, V and the perceptrons. The transformer is residual (frame “The transformer block in detail”): it is the same optimal control problem as for a ResNet.
- **Generation.** Sample $t_{n+1} \sim p_\theta(\cdot \mid t_1, \dots, t_n)$, append it to the sequence, start again.

Two toy experiments [Welleck 2025]

Where to look.

- Data: strings of length 10 to 15 over $\{0, 1, 2\}$ with exactly one 3: `11020103221`, `20232212010`, ...
- The only regularity is the uniqueness of the 3. To predict t_{i+1}, the only useful information is: *has the* 3 *already appeared?*
- Attention map after training: as soon as the 3 has appeared, every token looks at the position of the 3. The model has learned where to look.



Weights a_{ij}, the 3 in position 6 [Welleck 2025].

The attention map shows what the model has learned: not the next token, but where the information is.

Two toy experiments (continued) [Welleck 2025]

Counting letters.

- Data: a first name followed by its letter counts, `anna` → `a2n2`, `simons` → `s2i1m1o1n1`.
- On first names never seen: 97 % exact answers. The rule is learned, not the list.
- Out of distribution it breaks down: `aaa` → `ab1b1a2`, `s3an` → `a1a1a1a1a1a1a1a1`.

A transformer learns rules from sequences alone, but its abilities are irregular:
excellent near the data, unpredictable far from them.

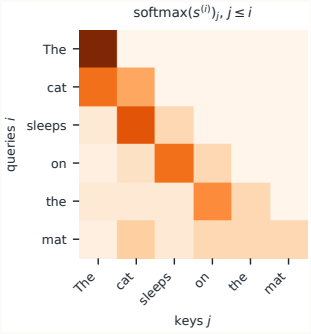
The transformer block in detail

$$\mathcal{T} = \left(\mathrm{Id} + \mathcal{M}_L^* \circ \mathcal{N}_{M,L}^*\right) \circ \left(\mathrm{Id} + \mathcal{A}_L \circ \mathcal{N}_{A,L}^*\right) \circ \ldots \circ \left(\mathrm{Id} + \mathcal{M}_1^* \circ \mathcal{N}_{M,1}^*\right) \circ \left(\mathrm{Id} + \mathcal{A}_1 \circ \mathcal{N}_{A,1}^*\right)$$

- $\mathrm{Id} + \cdots$: residual connections, exactly as in a ResNet. A transformer is a neural ODE on *sequences* of points of $\mathbb{R}^d$.
- $\mathcal{N}$: layer normalization, $\mathcal{N}(e) = \frac{\mathrm{diag}(s)}{\sigma_e}(e - \mu_e) + m$ (μ_e, σ_e: mean and standard deviation of the entries of e, $s, m \in \mathbb{R}^d$ learned). A star means "token by token".
- $\mathcal{M}$: perceptron $\mathcal{M} = W^{(d,D)} \circ \varrho^* \circ W^{(D,d)}$, $D \geq d$, with $W^{(a,b)}$ learned affine maps $\mathbb{R}^b \to \mathbb{R}^a$ and the GELU activation $\varrho(x) = x\,\Phi(x)$ (Φ: Gaussian distribution function).
- $\mathcal{A}$: the self-attention of the previous frames, followed by a learned linear map W_{out}:

$$\mathcal{A}(e)_i = W_{\mathrm{out}}\Big(\sum_{j \leq i} a_{ij}\, V e^{(j)}\Big), \qquad a_{ij} = \mathrm{softmax}_{j \leq i}\left(\left\langle Q e^{(i)}, K e^{(j)}\right\rangle / \sqrt{d_k}\right).$$

In practice h "heads" (Q_m, K_m, V_m) in parallel, of dimension $d_k = d/h$, concatenated before W_{out}.



Causal attention matrix a_{ij} (toy example, $s_j^{(i)}$: the scores): each token looks at the previous ones.

Transformers as a particle system

Forget the normalization and the perceptron. With an Euler step $1/L$ as in Section 8.1, an attention layer (non-causal, one head) reads

$$e^{(i)} \longmapsto e^{(i)} + \frac{1}{L} \sum_{j=1}^{n} \frac{\exp\big(\langle Qe^{(i)}, Ke^{(j)} \rangle\big)}{\sum_k \exp\big(\langle Qe^{(i)}, Ke^{(k)} \rangle\big)} Ve^{(j)}.$$

Limit $L \to \infty$: n interacting particles $x_i(t) \in \mathbb{R}^d$,

$$\dot{x}_i = \sum_{j=1}^{n} \frac{\exp\big(\langle Qx_i, Kx_j \rangle\big)}{\sum_k \exp\big(\langle Qx_j, Kx_k \rangle\big)} Vx_j, \qquad i \in [\![1, n]\!],$$

a control system (controls Q, K, V) of *mean-field* type, to be compared with the neural ODE $\dot{x} = F(x, u)$.

- [Geshkovski, Letrouit, Polyanskiy, Rigollet 2023]: on the sphere, with $Q = K = V = \mathrm{Id}$, the particles cluster as $t \to \infty$.
- [Geshkovski, Rigollet, Ruiz-Balet 2024]: controllability (interpolation of measures). [Alcalde, Fantuzzi, Zuazua 2024]: clustering and classification with *hardmax* attention.
- Controllability, approximation of diffeomorphisms, optimal control: the same questions as in chapter 8 arise, at a larger scale.

Transformers for mathematics [Welleck 2025]

Computations

- 4-digit addition: trained on $5 \cdot 10^6$ of the 10^8 pairs, 100 % success on the others.
- Symbolic integration, ODEs of order 1 and 2 [Lample, Charton 2019]: integrals that Mathematica does not find.
- Stability of differential systems [Charton et al. 2021], global Lyapunov functions [Alfarano et al. 2024].

Discovery

- Triangle-free graphs on n vertices with as many edges as possible: local search, then a transformer trained on the best 10 % of the graphs, then local search again from its outputs.
- PatternBoost [Charton, Ellenberg, Wagner, Williamson 2024] alternates these two phases. Counterexample to the Graham and Harary conjecture (1992) on the hypercube.

Two conditions: the problem must be verifiable (differentiate the integral, count the triangles) and one needs a data generator. Many candidates are produced and the wrong ones are discarded.

The notebooks of [Welleck 2025] train a transformer of a few million parameters in a few minutes: `github.com/wellecks/transformers4math-simons`.

Perspectives: the abundance of proofs [Tao 2026]

What happened since 2024.

- IMO 2024: AlphaProof (DeepMind) at silver medal level, with proofs written in Lean.
  IMO 2025: models from DeepMind and OpenAI at gold medal level, in natural language, without a formal tool.
  Then AlphaEvolve (2025) and Erdős problems (2025–26), among them the unit distances.
- September 8, 2026: OpenAI announces finite-time blow-up for 3D Navier–Stokes with smooth forcing, a proof formalized in Lean, obtained by ~ 10^4 agents in 88 hours, a few hours after Buckmaster and Alpöge, following the strategy of Córdoba and Martínez-Zoroa.
  Verification in progress, the case without forcing still open.
- September 21, 2026: OpenAI claims to have solved more than 100 open problems. Open letter “Release the proofs”.
  October 6: 719 manuscripts in 372 families, 17 fields, published on `github.com/openai/math`, ~ 3 hours of computation per result, 42 % of the main statements formalized in Lean.
  The next day, 3 manuscripts withdrawn for a sign error and 14 corrected.

Figures as reported in [Tao 2026] and on `github.com/openai/math`, October 2026.

Perspectives: why, without changing the architecture? [Tao 2026]

The transformer block is the one of 2017. What changed:

- scale: data, parameters, computation.
- post-training: reinforcement learning with a *verifiable reward*, a final answer or a Lean checker: generate, verify, keep.
- inference-time computation: long chains of reasoning, massive sampling of candidates, verification.

[Tao 2026] Mathematics is the ideal playground: proofs are objectively verifiable, digitizable and available in large quantity. But the abilities remain *irregular*, failures are not reported, a Lean statement still has to be read by a human, and "Math 2.0" asks to keep human understanding at the center.

What to remember

Machine learning

- ML = function approximation with a nonlinearly parametrized family.
- Universal approximation: any reasonable nonlinearity works (Stone–Weierstrass).
- Backpropagation: the chain rule organized backwards, cost ≈ 2× an evaluation.
- Depth ⇒ vanishing gradients ⇒ ResNets.

Control theory

- ResNet = explicit Euler for $\dot{x} = F(x, u)$.
- Expressivity = controllability: simultaneous controllability and universal approximation at width $2d' + 1$ (Tabuada and Gharesifard), proved with geometric control (Lie brackets).
- Learning = optimal control (8.17). Exact memorization is neither constructive nor desirable: regularize, as in (8.21).

The dictionary ResNet ↔ control is not only an analogy: it provides theorems (expressivity, memorization) and open questions (rates, architectures, transformers).

References

- D. Prandi, *Optimisation, Control, and Data*, lecture notes, 2026. (Chapters 6–8.)
  https://dprn.github.io/docs/lecture-notes/poly-ocd.pdf
- P. Petersen, J. Zech, *Mathematical Theory of Deep Learning*, 2025. (Universal approximation.)
  https://arxiv.org/abs/2407.18384
- A. Ng, T. Ma, *CS229 Lecture Notes*, Stanford. (Support vector machines, kernels.)
  https://cs229.stanford.edu/main_notes.pdf
- P. Tabuada, B. Gharesifard, *Universal approximation power of deep residual neural networks through the lens of control*, IEEE Trans. Automat. Control 68(5), 2023. (Theorems 8.1.2 to 8.1.4.)
  https://arxiv.org/abs/2007.06007
- A. Scagliotti, *Deep learning approximation of diffeomorphisms via linear-control systems*, Math. Control Relat. Fields 13(3), 2023. (The ResNet / control dictionary, the regularized cost (8.21).)
  https://arxiv.org/abs/2110.12393
- S. Frieder et al., *Large language models for mathematicians*, 2024. (Transformers.)
  https://arxiv.org/abs/2312.04556
- S. Welleck, *Transformers for mathematics*, tutorial, Simons Institute and SLMath, 2025. (Attention, toy experiments, notebooks.)
  https://github.com/wellecks/transformers4math-simons

References (epilogue)

- T. Tao, *Math 2.0*, talk at Caltech, October 2026. (Perspectives.)
  https://teorth.github.io/tao-web/slides/math-2-0-caltech-2026.pdf
- Transformers as particle systems: B. Geshkovski, C. Letrouit, Y. Polyanskiy, P. Rigollet, *A mathematical perspective on transformers*, 2023; B. Geshkovski, P. Rigollet, D. Ruiz-Balet, *Measure-to-measure interpolation using transformers*, 2024; A. Alcalde, G. Fantuzzi, E. Zuazua, *Clustering in pure-attention hardmax transformers and its role in sentiment analysis*, 2024.
  https://arxiv.org/abs/2312.10794 · https://arxiv.org/abs/2411.04551 · https://arxiv.org/abs/2407.01602
- Transformers for mathematics: G. Lample, F. Charton, *Deep learning for symbolic mathematics*, 2019; F. Charton, A. Hayat, G. Lample, *Learning advanced mathematical computations from examples*, 2021; A. Alfarano, F. Charton, A. Hayat, *Global Lyapunov functions: a long-standing open problem in mathematics, with symbolic transformers*, 2024; F. Charton, J. Ellenberg, A. Wagner, G. Williamson, *PatternBoost: constructions in mathematics with a little help from AI*, 2024.
  https://arxiv.org/abs/1912.01412 · https://arxiv.org/abs/2006.06462 · https://arxiv.org/abs/2410.08304 · https://arxiv.org/abs/2411.00566

Questions: dario.prandi@centralesupelec.fr